
Metaheurísticas

Seminario 3. Problemas de optimización con técnicas basadas en poblaciones

1. Estructura de un Algoritmo Genético/Memético y Aspectos de Implementación
1. Problemas de Optimización con Algoritmos Genéticos y Meméticos
 - Problema del Portfolio
 - Aprendizaje de Pesos en Características

Estructura de un Algoritmo Genético

Procedimiento Algoritmo Genético

Inicio (1)

$t = 0;$

inicializar $P(t);$

evaluar $P(t);$

Mientras (no se cumpla la condición de parada) hacer

Inicio(2)

$t = t + 1$

seleccionar P' desde $P(t-1)$

recombinar P'

mutar P'

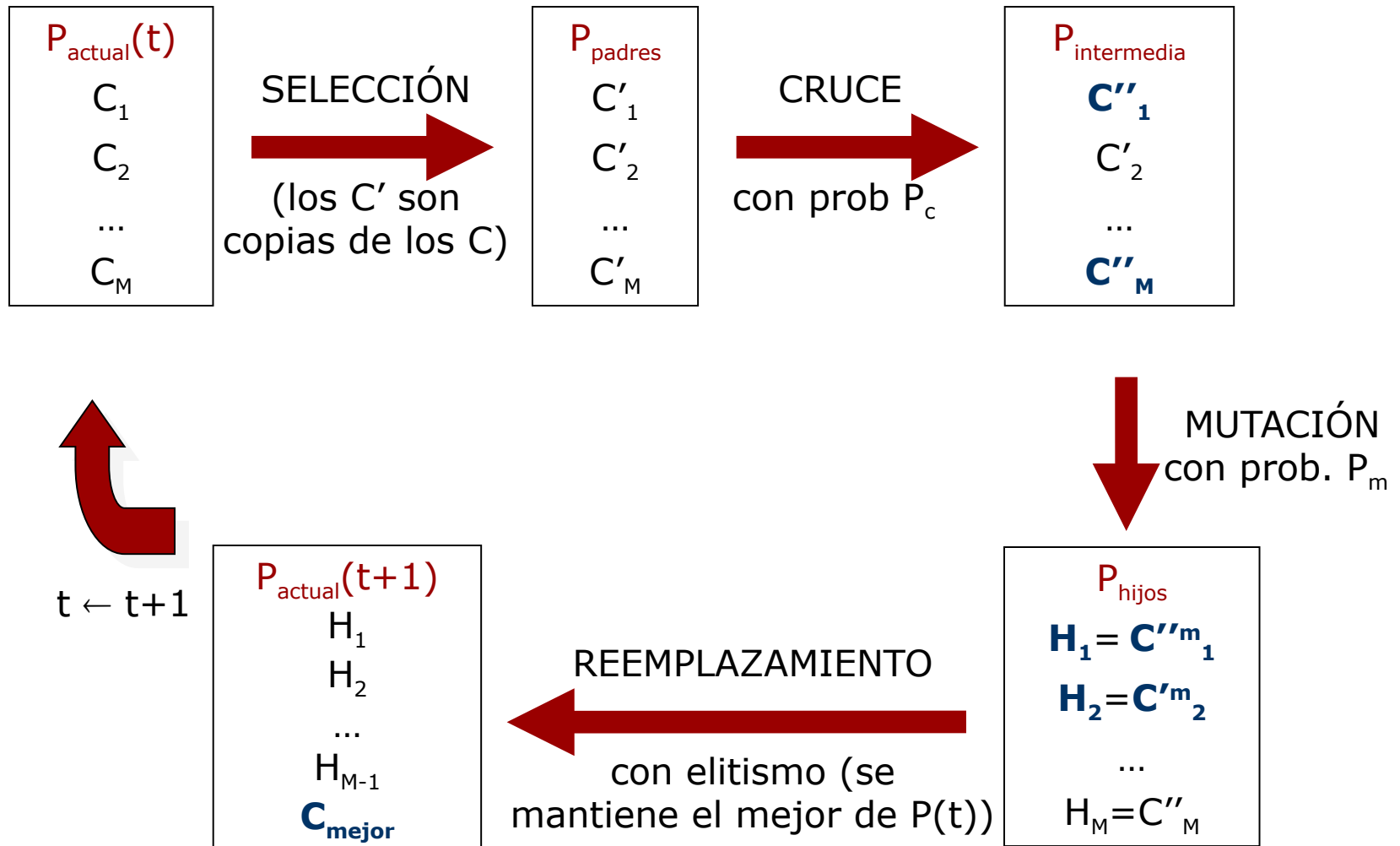
reemplazar $P(t)$ a partir de $P(t-1)$ y P'

evaluar $P(t)$

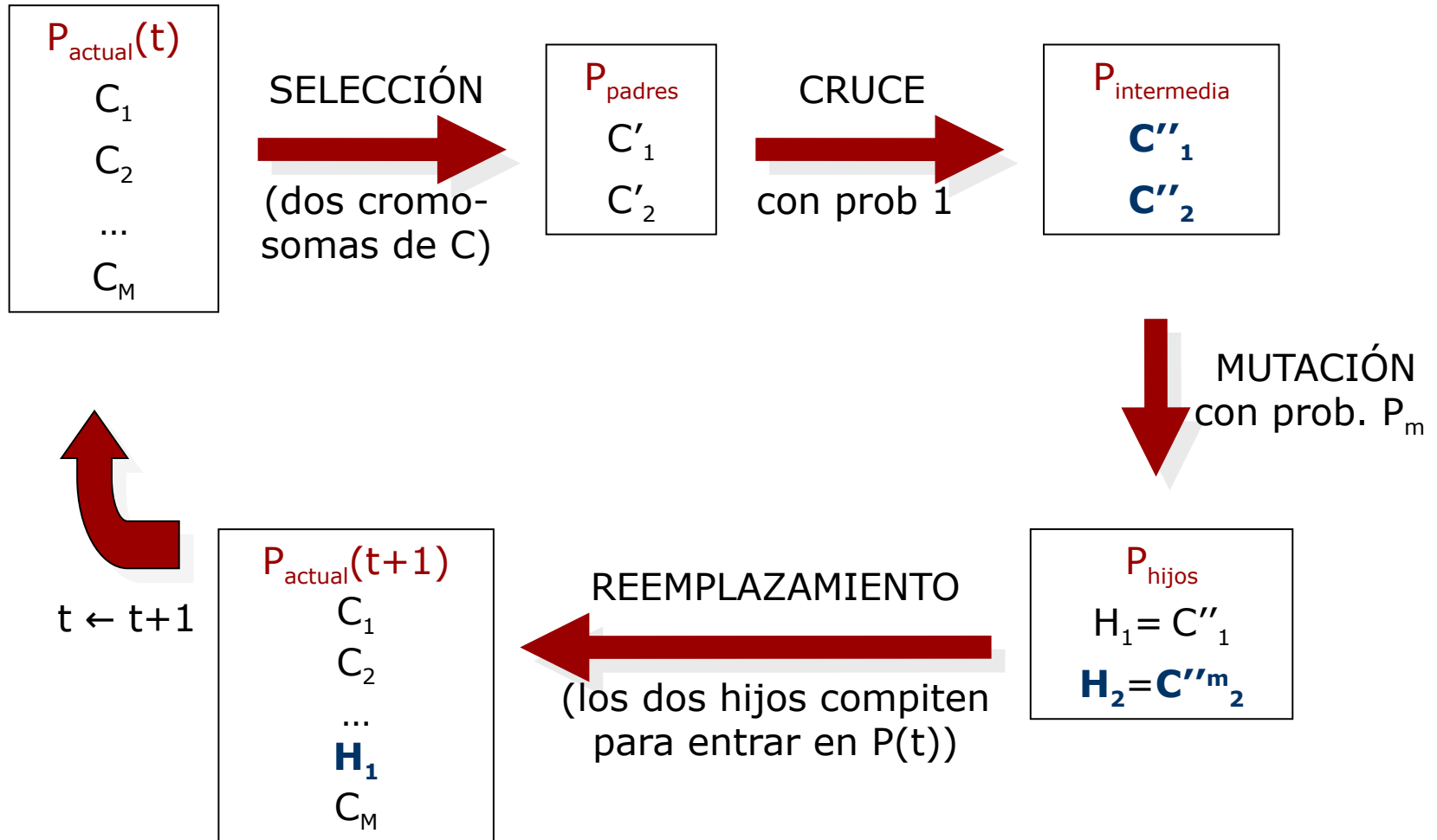
Final(2)

Final(1)

Modelo Generacional



Modelo Estacionario



Modelo Generacional:

Aspectos de Implementación

- ✓ Lo mas costoso en tiempo de ejecución de un Algoritmo Genético es la generación de números aleatorios para:
 - ✓ Aplicar el mecanismo de selección
 - ✓ Emparejar las parejas de padres para el cruce
 - ✓ Decidir si una pareja de padres cruza o no de acuerdo a P_c
 - ✓ **Decidir si cada gen muta o no de acuerdo a P_m**
- ✓ Se pueden diseñar implementaciones eficientes que reduzcan en gran medida la cantidad de números aleatorios necesaria:
 - ✓ Emparejar las parejas para el cruce: Como el mecanismo de selección ya tiene una componente aleatoria, se aplica siempre un emparejamiento fijo: el primero con el segundo, el tercero con el cuarto, etc.

Modelo Generacional:

Aspectos de Implementación

- ✓ Decidir si una pareja de padres cruza: En vez de generar un aleatorio u en $[0,1]$ para cada pareja y cruzarla si $u \leq P_c$, se estima a priori (al principio del algoritmo) el número de cruces a hacer en cada generación (**esperanza matemática**):

$$N^o \text{ esperado cruces} = P_c \cdot M/2$$

- ✓ Por ejemplo, con una población de 60 cromosomas (30 parejas) y una P_c de 0.6, cruzarán $0,6 \cdot 30 = 18$ parejas
- ✓ De nuevo, consideramos la aleatoriedad que ya aplica el mecanismo de selección y cruzamos siempre las $N^o \text{ esperado cruces}$ primeras parejas de la población intermedia

Modelo Generacional:

Aspectos de Implementación

- ✓ Decidir si cada gen muta: El problema es similar al del cruce, pero mucho mas acusado
- ✓ Normalmente, tanto el tamaño de población M como el de los cromosomas n es grande. Por tanto, el número de genes de la población, $M \cdot n$, es muy grande
- ✓ La P_m , definida a nivel de gen, suele ser muy baja (p.e. $P_m=0.01$). Eso provoca que se generen muchos números aleatorios para finalmente realizar muy pocas mutaciones
- ✓ Por ejemplo, con una población de 60 cromosomas de 100 genes cada uno tenemos 6000 genes de los cuales mutarían unos 60 (*Nº esperado mutaciones* = $P_m \cdot n^\circ \text{ genes población}$, **esperanza matemática**)
- ✓ Generar 6000 números aleatorios en cada generación para hacer sólo 60 mutaciones (en media) es un gasto inútil. Para evitarlo, haremos siempre exactamente *Nº esperado mutaciones* en cada generación

Modelo Generacional:

Aspectos de Implementación

- ✓ Aparte de hacer un número fijo de mutaciones, hay que decidir cuáles son los genes que mutan
- ✓ Normalmente, eso se hace también generando números aleatorios, en concreto dos, un entero en $\{1, \dots, M\}$ para escoger el cromosoma y otro en $\{1, \dots, n\}$ para el gen
- ✓ Existen también mecanismos más avanzados que permiten escoger el gen a mutar generando un único número real en $[0,1]$ y haciendo unas operaciones matemáticas (ver código entregado en prácticas)

Aspectos de Diseño de los Algoritmos Meméticos

- Una decisión fundamental en el diseño de un Algoritmo Memético (AM) es la definición del equilibrio entre:
 - la exploración desarrollada por el algoritmo de búsqueda global (el Algoritmo Genético (AG)) y
 - la explotación desarrollada por el algoritmo de búsqueda local (BL)
- La especificación de este **equilibrio entre exploración y explotación** se basa principalmente en dos decisiones:
 1. ¿Cuándo se aplica el optimizador local
 - En cada generación del AG o
 - cada cierto número de generacionesy sobre qué agentes?
 - Sólo sobre el mejor individuo de la población en la generación actual o
 - sobre un subconjunto de individuos escogidos de forma fija (los m mejores de la población) o variable (de acuerdo a una probabilidad de aplicación p_{LS})

Aspectos de Diseño de los Algoritmos Meméticos

3. ¿Sobre qué agentes se aplica (anchura de la BL) y con qué intensidad (profundidad de la BL)?
- AMs baja intensidad (alta frecuencia de aplicación de la BL/pocas iteraciones)
 - AMs alta intensidad (baja frecuencia de la BL/muchas iteraciones)

Problema del Portfolio



¿En qué acciones invertir
nuestro dinero?

- Queremos:
 - Rentabilidad.
 - Diversificar.
 - Reducir Riesgo.



A. Salo, M. Doumpos, J. Liesiö, y C. Zopounidis, «Fifty years of portfolio optimization», European Journal of Operational Research, vol. 318, n.o1, pp. 1-18, oct. 2021, doi: 1.

Problema del Portfolio

- Para ello se usa un vector de números reales que debe de cumplir que:

$$\sum_{i=1}^N w_i = 1 \quad (w_i = 0) \vee (w_i \geq \text{minvalue} \wedge w_i \leq \text{maxvalue}) \quad \forall i \in [1, N]$$

- Y se calcula fitness como:

$$\text{Beneficio}(w) = \sum w_i \cdot \text{Ben}_i, \text{ donde } \text{Ben}_i = \sum_t^T \log(1 + \mu_i)$$

$$\text{Riesgo}(w) = \sqrt{\sum_i \sum_j w_i \cdot w_j \text{cov}(i, j)} \quad \text{cov}(i, j) = \frac{\sum_t (\mu_i - \bar{\mu}_i)(\mu_j - \bar{\mu}_j)}{T}$$

$$\text{Fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

Representación Genético

- Representación Real: Cada valor representa el porcentaje.

Para que la representación sea factible, debe cumplir siempre todas las restricciones.

- Generación de la población inicial: aleatoria verificando las restricciones
- Modelos de evolución: 2 variantes: generacional con elitismo / estacionario con 2 hijos que compiten con los dos peores de la población
- Mecanismo de selección: torneo con $k=3$
- Operador de mutación: Intercambio. Se intercambia el 15% del valor del gen x_i por el de otro gen x_j escogido aleatoriamente con el valor contrario, respetando las restricciones.

Algoritmo Genético para el Portfolio

Operador de Cruce 1: Cruce Aritmetico aleatorio

- Genera un hijo a partir de dos padres. Para generar dos hijos, lo ejecutaremos dos veces a partir de los mismos padres
- Aquellas posiciones que contengan el mismo valor en ambos padres se mantienen en el hijo (para preservar las selecciones prometedoras)
- Se escoge para cada par de padres y posicion i un valor σ_i entre $[0, 1]$
- Para el resto de posiciones se hace:

$$Hijo_i^1 = \sigma_i \cdot Padre_i^1 + (1 - \sigma_i) \cdot Padre_i^2$$

$$Hijo_i^2 = \sigma_i \cdot Padre_i^2 + (1 - \sigma_i) \cdot Padre_i^1$$

Cuidado: Hay que validar las restricciones

Algoritmo Genético para el Portfolio

Cruce basado en el cruce aritmético aleatorio

x1	x2	x3	x4	x5
----	----	----	----	----

y1	y2	y3	y4	y5
----	----	----	----	----

$\alpha_1 = \text{Random}(0.0, 1.0)$ 
(α_1 es elegido aleatoriamente para cada posición)

Sol ₁	$\alpha_1 x_1 + (1 - \alpha_1) y_1$	$\alpha_2 x_2 + (1 - \alpha_2) y_2$	$\alpha_3 x_3 + (1 - \alpha_3) y_3$	$\alpha_4 x_4 + (1 - \alpha_4) y_4$	$\alpha_5 x_5 + (1 - \alpha_5) y_5$
Sol ₂	$\alpha_1 y_1 + (1 - \alpha_1) x_1$	$\alpha_2 y_2 + (1 - \alpha_2) x_2$	$\alpha_3 y_3 + (1 - \alpha_3) x_3$	$\alpha_4 y_4 + (1 - \alpha_4) x_4$	$\alpha_5 y_5 + (1 - \alpha_5) x_5$

Algoritmo Genético para el Portfolio

Cruce BLX- α con $\alpha=0.3$

- Dados 2 cromosomas

$$C1 = (c1_1, \dots, c1_n) \text{ y } C_2 = (c_{21}, \dots, c_{2n}),$$

- BLX- α genera dos descendientes

$$H_k = (h_{k1}, \dots, h_{ki}, \dots, h_{kn}), k = 1, 2,$$

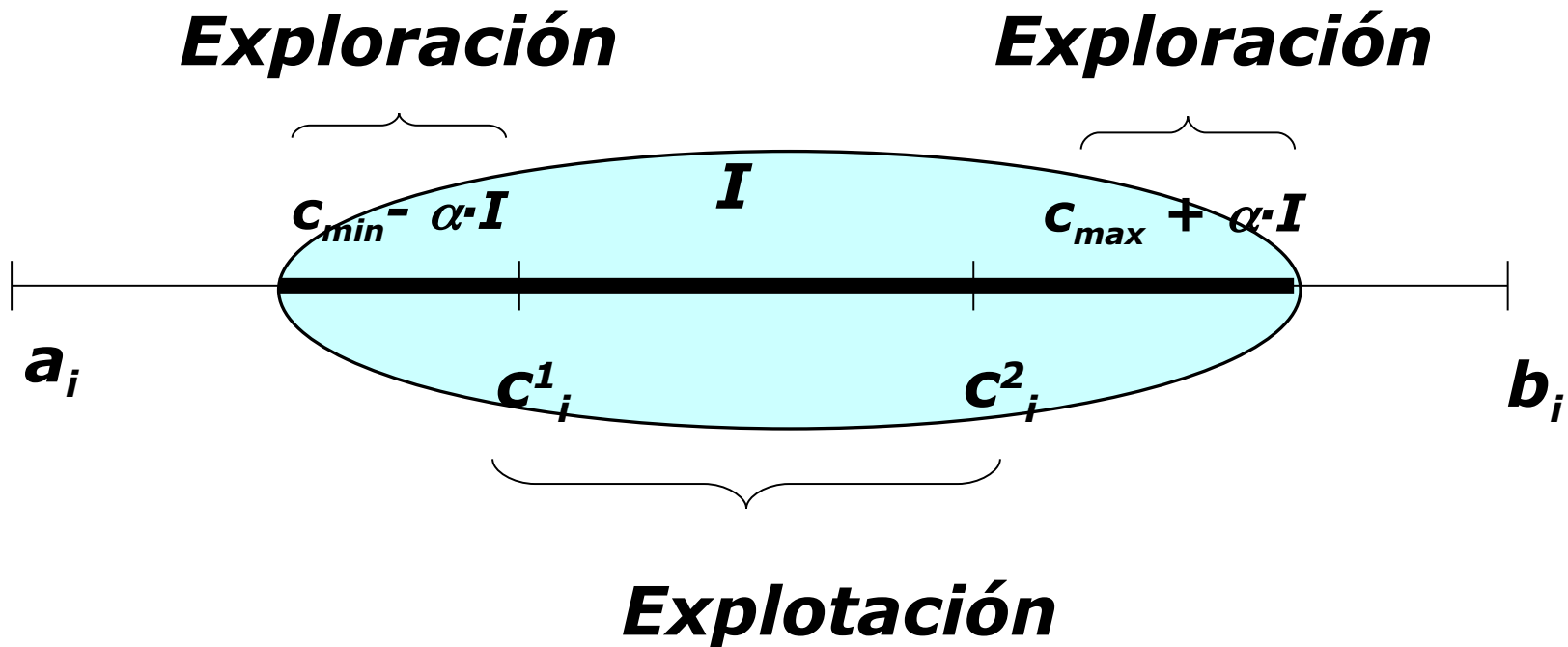
- donde h_{ki} se genera aleatoriamente en el intervalo:

$$[C_{\min} - l \cdot \alpha, C_{\max} + l \cdot \alpha]$$

- $C_{\max} = \max \{c1_i, c_{2i}\}$
- $C_{\min} = \min \{c1_i, c_{2i}\}$
- $l = C_{\max} - C_{\min}, \alpha \in [0, 1]$

Algoritmo Genético para el Portfolio

Cruce BLX- α con $\alpha=0.3$



$Sol_1[i] = \text{aleatorio en el rango } [C_{min}-\alpha \cdot I, C_{max}+\alpha \cdot I]$

$Sol_2[i] = \text{aleatorio en el rango } [C_{min}-\alpha \cdot I, C_{max}+\alpha \cdot I]$

Algoritmo Memético para el Portfolio

Con el objetivo de no desbalancear demasiado el equilibrio exploración-explotación emplearemos una BL para incorporarla al esquema de los AGs.

Se busca que sea menos disruptiva que la BL funcionando por separado.

Aplicamos el esquema de la Búsqueda Local de la Práctica 1 pero desplazando un 15% del valor, en vez del 40% de la práctica 1.

Problema del agrupamiento con restricciones (PAR)

El Problema del Agrupamiento con Restricciones (PAR) bajo **restricciones débiles** consiste en minimizar una agregación de (número de restricciones incumplidas) y (la desviación general) permitiendo

Para abordar este problema mediante AGs debemos definir la representación del mismo, así como los operadores propios de esta metaheurística

Algoritmo Genético para el PAR: Representación de la solución

Emplearemos el modelo de representación basado en etiquetas que ya utilizamos para la BL de la primera práctica. Cada individuo de la población consiste en un vector de posiciones en correspondencia con las instancias del conjunto de datos . En cada posición se almacena la etiqueta de la instancia asociada a dicha posición:

$$\begin{array}{cccccccc} & x_0 & x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \\ C = & \{1, & 1, & 1, & 1, & 2, & 2, & 2, & 2\} \\ & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ & s_0 & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 \\ S = & [1, & 1, & 1, & 1, & 2, & 2, & 2, & 2] \end{array}$$

AG para el PAR: Función Objetivo

De nuevo emplearemos la misma función objetivo que en la práctica 1. Con el mismo método para el cálculo del parámetro .

$$f = \overline{C} + (infeasibility * \lambda)$$

Desviación General

Número De Restricciones Incumplidas

AG para el PAR: Aspectos Generales

- **Generación de la población inicial:** aleatoria, para cada cromosoma generamos valores entre 0 y $k-1$, **verificando las restricciones**
- **Modelos de evolución:** 2 variantes: generacional con elitismo/estacionario con 2 hijos que compiten con los dos peores de la población
- **Mecanismo de selección:** torneo con 3 individuos
- **Operador de cruce:** 2 variantes, que generan un único hijo por cada pareja de padres. Se aplican 2 veces sobre cada pareja de padres para obtener 2 descendientes
Ambos **pueden generar soluciones no factibles** por dejar algún clúster vacío. En ese caso, se aplicaría una reparación, moviendo un elemento escogido aleatoriamente a dicho cluster

AG para el PAR: Operador de cruce uniforme

El operador de cruce uniforme produce un nuevo individuo (hijo) en base a dos individuos dados (padres) combinando de manera uniforme las características de los dos padres

Para ello se seleccionan aleatoriamente la mitad de los genes de un padre y la otra mitad de otro padre

Procedimiento: generamos números aleatorios distintos en el rango. Seleccionamos uno de los padres y copiamos en la descendencia los genes cuyo índice coincide con los números generados. Los genes que quedan por asignar en la descendencia se copian del segundo padre.

El orden de primer y segundo padre se intercalan para cada uno de los hijos.

AG para el PAR: Operador de cruce uniforme

El operador de cruce uniforme produce un nuevo individuo (hijo) en base a dos individuos dados (padres) combinando de manera uniforme las características de los dos padres

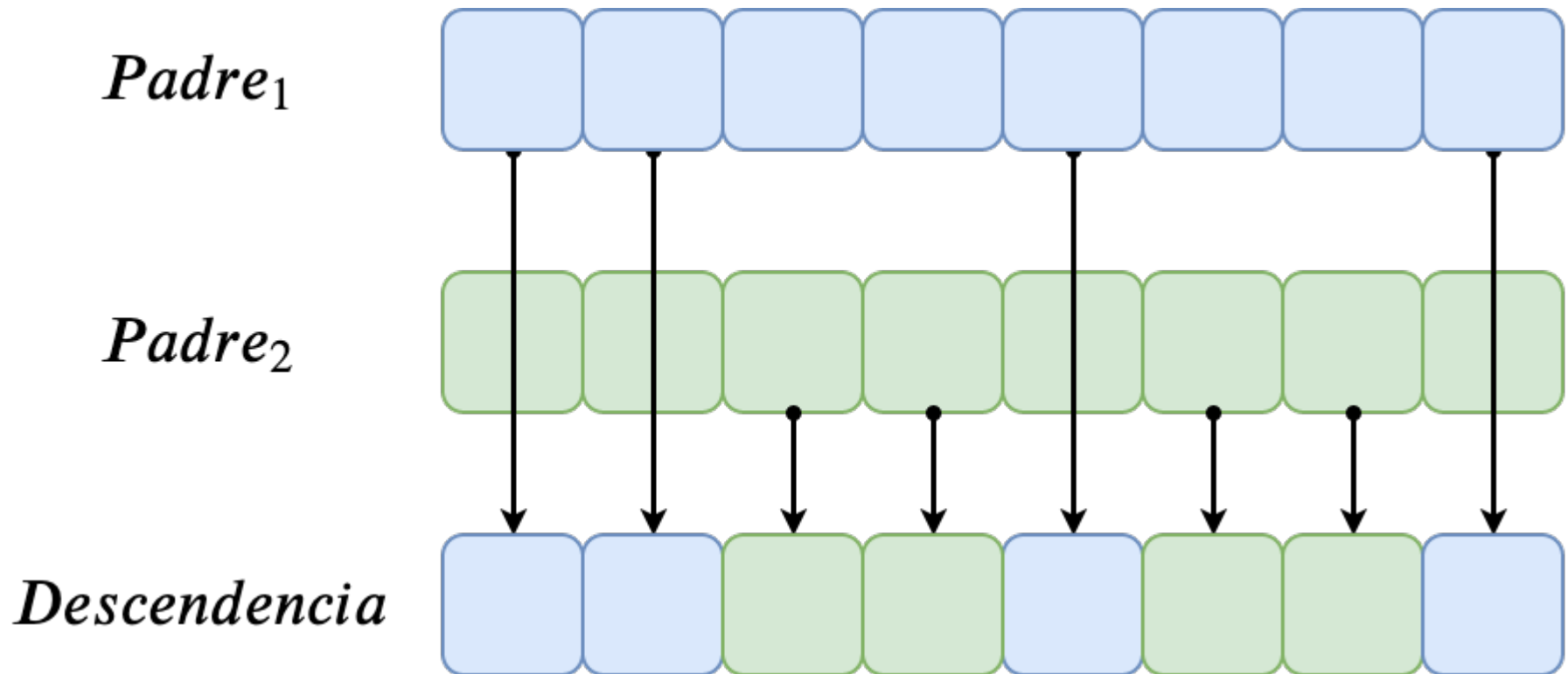
Para ello se seleccionan aleatoriamente la mitad de los genes de un padre y la otra mitad de otro padre

Procedimiento: generamos números aleatorios distintos en el rango. Seleccionamos uno de los padres y copiamos en la descendencia los genes cuyo índice coincide con los números generados. Los genes que quedan por asignar en la descendencia se copian del segundo padre.

El orden de primer y segundo padre se intercalan para cada uno de los hijos.

AG para el PAR: Operador de cruce uniforme

- Ejemplo para $n=8$. Obtenemos la lista de números aleatorios $\{0, 1, 1, 7\}$. Tomamos el primer progenitor como $Padre_1$:



AG PAR: Operador de cruce por segmento fijo

El operador de cruce por segmento fijo produce un nuevo individuo (hijo) en base a dos individuos dados (padres) seleccionando de uno de ellos un segmento continuo de características y copiándolo sin modificación a la descendencia

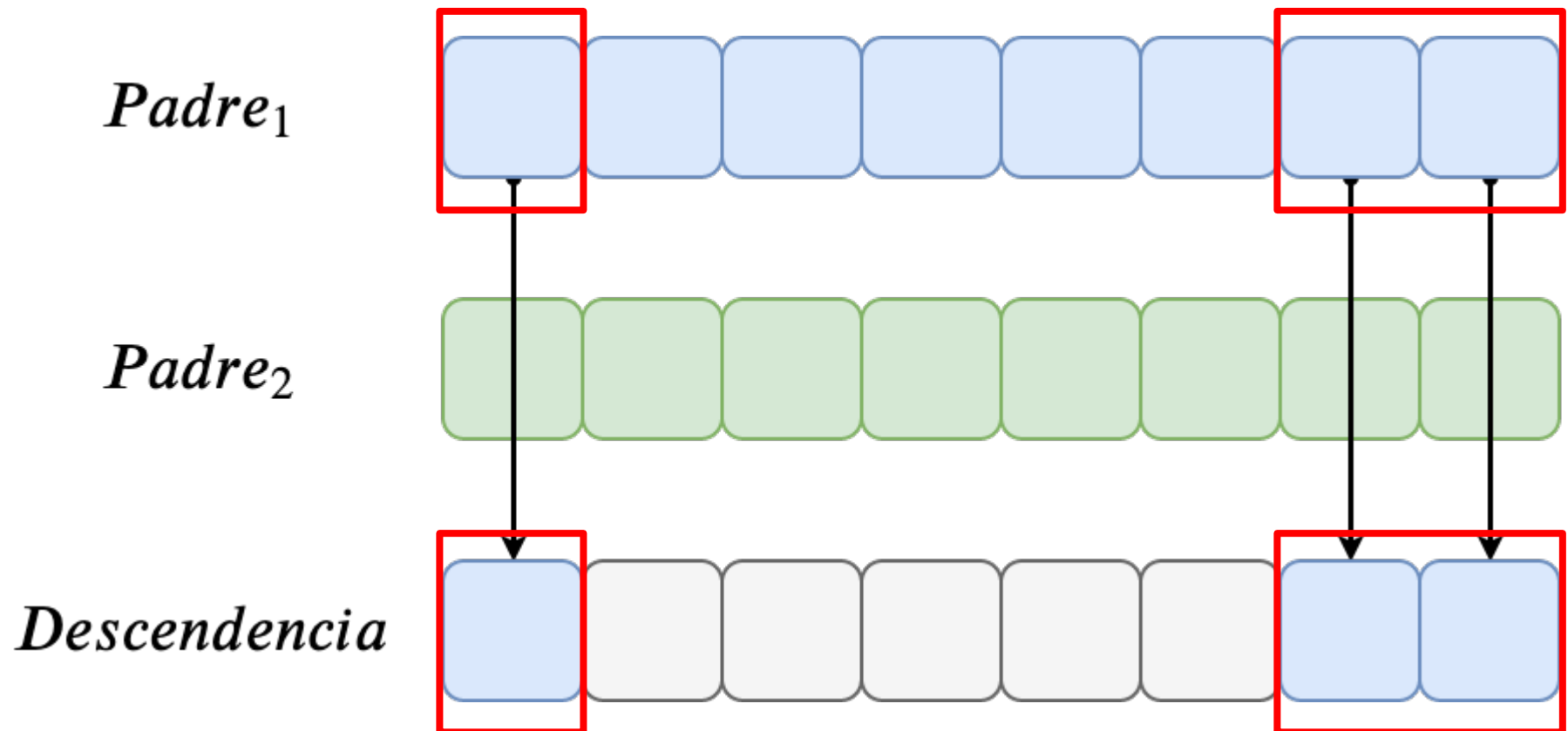
Los genes que quedan por asignar en la descendencia combinan de manera uniforme características de los dos padres

La posición y el tamaño de segmento se generan aleatoriamente para cada aplicación del cruce (el mismo para ambos hijos)

En cada cruce se generan dos números aleatorios u (inicio del segmento) en el rango $[0, N-1]$ y v (tamaño del segmento) en el rango $[1, N/3]$. Seleccionamos un padre y copiamos a la descendencia los genes con los índices en el intervalo del segmento. El resto de genes de la descendencia se determinan igual que en el cruce uniforme

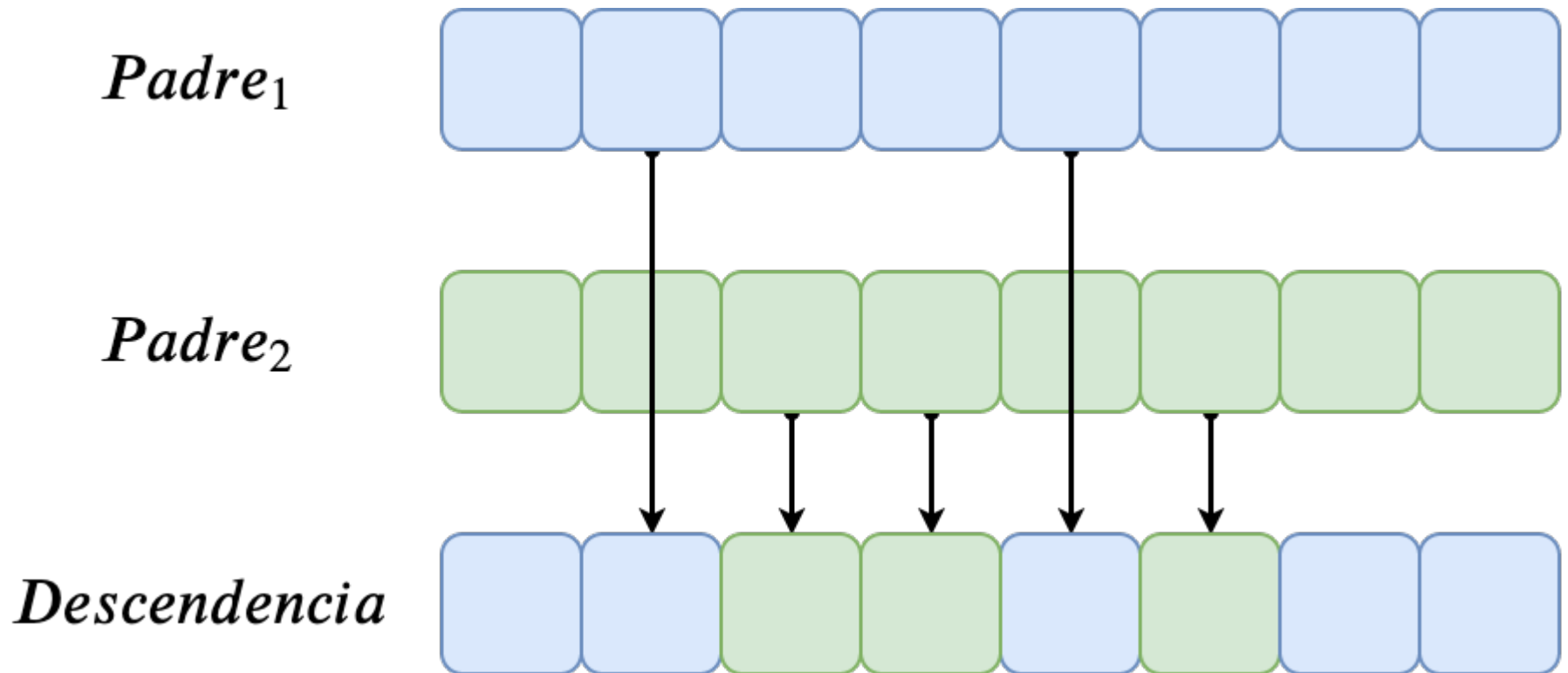
AG PAR: Operador de cruce por segmento fijo

- Ejemplo para $n=8$ y $v=3$. Generamos $r=6$. Calculamos el final del segmento como $((r+1) \bmod n) - 1 = ((6+1) \bmod 8) - 1 = 0$. Tomamos el primer progenitor como Padre_1 :



AG PAR: Operador de cruce por segmento fijo

El resto de genes se seleccionan de manera uniforme de entre los dos padres. Generamos la lista de números aleatorios $\{1,1\}$:



AG PAR: Operador de cruce por segmento fijo

¡El operador de cruce por segmento fijo está sesgado!

Siempre se seleccionan más genes del padre que se elige como portador del segmento

Si seleccionamos como padre portador del segmento el mejor de los dos estaremos **favoreciendo la explotación**, de forma que la población converge rápidamente. Por eso es bueno que cada uno de los hijos use un padre portador diferente, que es lo que planteamos.

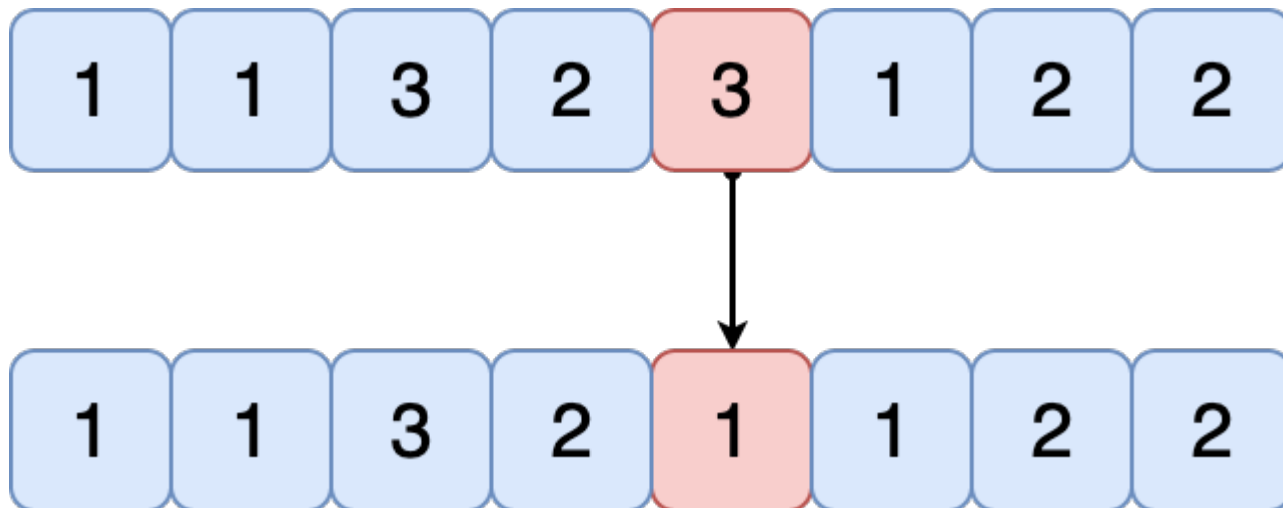
Si se seleccionase como padre portador del segmento el peor padre estaremos **favoreciendo la exploración** y la población tardará más en converger

(Extra: Puede ser interesante hacer un estudio de convergencia de la población variando parámetros del operador de cruce de segmento fijo)

AG PAR: Operador de mutación uniforme

Emplearemos el operador de mutación uniforme

Supongamos que hemos determinado que debe mutar un gen por cada cromosoma en la población. Basta con generar dos números aleatorios, uno para determinar el gen que muta (en el intervalo) y otro para determinar el nuevo valor (que debe ser distinto del actual **y mantener las restricciones**). Supongamos que muta el gen 5 y que su nuevo valor es 1. El cambio es tan simple como:



Algoritmo Memético para el PAR: Búsqueda Local Suave

Con el objetivo de no desbalancear demasiado el equilibrio exploración-explotación emplearemos una BL suave para incorporarla al esquema de los AGs

Dado un cromosoma, consiste en seleccionar elementos del mismo de forma aleatoria (sin repetición) y asignar a dichos elementos el mejor valor posible con la información disponible

Recorremos el cromosoma una sola vez

AM para el PAR: Búsqueda Local Suave

Algorithm 1 Búsqueda Local Suave, BLS

```
1: function BLS( $S$ ,  $bestfit$ ,  $maxevals$ ,  $\epsilon$ )
2:    $RSI \leftarrow RandomShuffle(1, \dots, n)$ 
3:    $fallos \leftarrow 0$ 
4:    $i \leftarrow 0$ 
5:    $newsol \leftarrow S$ 
6:   while ( $fallos < \epsilon$ )  $\wedge$  ( $i < N$ )  $\wedge$  ( $evals < maxevals$ ) do
7:      $p \leftarrow RSI[i]$ 
8:      $oldvalue \leftarrow sol[p]$ 
9:     for  $val$  in  $1..k$  and  $evals < maxevals$  do
10:       $newsol[p] \leftarrow val$ 
11:       $fit \leftarrow fitness(newsol)$ 
12:       $evals \leftarrow evals + 1$ 
13:      if  $fit < bestfit$  then
14:         $S \leftarrow newsol$ 
15:         $bestfit \leftarrow fit$ 
16:      else
17:         $newsol[p] \leftarrow S[p]$ 
18:      end if
19:    end for
20:    if  $oldvalue = newsol[p]$  then
21:       $fallos \leftarrow fallos + 1$ 
22:    end if
23:     $i \leftarrow i + 1$ 
24:  end while
25:  return  $S$ ,  $bestfit$ ,  $evals$ 
26: end function
```

PAR: Búsqueda Local Suave

Cuando la BL no produce cambios en el cromosoma decimos que falla

El contador de fallos está diseñado para evitar que se desperdicien evaluaciones de la función objetivo en cromosomas de mucha calidad. Permitiremos como mucho ξ fallos en la BL

La condición de parada asegura que si el cromosoma es localmente optimizable, como mucho se recorre una única vez, preservando hasta cierto punto el equilibrio exploración-explotación

Si no lo es, el contador de fallos crecerá rápidamente y la BL se detendrá

Problemas de Optimización con Algoritmos Meméticos

- En los dos problemas (Portfolio y PAR), emplearemos un AM consistente en un AG generacional que aplica una BL a cierto número de cromosomas cada cierto tiempo
- **Para el PAR emplearemos una nueva BL, no la implementada para la práctica 1**
- Se aplicará **una BL de baja intensidad**. En ambos **se evaluarán sólo 100 vecinos en total en cada aplicación** (en PAR se evaluara además el parámetro ξ y que solo se evalúe cada posición una vez).
- Se estudiarán las siguientes tres posibilidades de hibridación:
 - **AM-All**: Cada ciertas generaciones, aplicar la BL sobre **todos los cromosomas** de la población
 - **AM-Rand**: Cada ciertas generaciones, aplicar la BL sobre un **subconjunto de cromosomas** de la población seleccionado aleatoriamente con probabilidad p_{LS} igual al **0.1** para cada cromosoma
 - **AM-Best**: Cada ciertas generaciones, aplicar la BL sobre los **0.1·N mejores** cromosomas de la población actual (N es el tamaño de ésta)

NOTA SOBRE LOS TIEMPOS DE EJECUCIÓN

En los AGs de la segunda práctica no es posible emplear ninguna factorización de la función objetivo

Esto se debe a que los operadores de cruce empleados provocan un cambio significativo en las soluciones candidatas generadas y es necesario evaluarlas de forma estándar

Eso puede provocar que las ejecuciones de los AGs sean más lentas que las de la BL si éstas se optimizaron.

Además, en esta práctica existe un mayor número de algoritmos

Hay que tener estos hechos en cuenta y no dejar las ejecuciones para el último momento.